Laboratorio segundo corte: Modelado y Análisis de Datos con SQL

Nayerlis Rosa Miranda Vega, Laura Vanessa Barrero, Sonia Andrea Sonza Rubiano, Zharick Dayana Peña Gualdron
Universidad De San Buenaventura Bogotá D.C., Colombia
*C*orreo: nrmirandav@academia.usbbog.edu.co, lvbarreroa@academia.usbbog.edu.co, sasonzar@academia.usbbog.edu.co, zdpenag@academia.usbbog.edu.co

***Resumen*—En este laboratorio se aborda el proceso de modelado y análisis de un conjunto de datos relacionado con streamers, con el propósito de transformar la información original en una base de datos relacional consistente. Para ello, se aplican las tres primeras formas normales y se implementa el modelo en MySQL, utilizando Node.js para la carga de datos. Finalmente, se ejecutan consultas SQL para analizar los juegos transmitidos, los idiomas, los seguidores y la audiencia de los streamers.**

***Index Terms*—Base de datos relacional, normalización, MySQL, Node.js, streamers.**

I. Introducción

El diseño y la gestión de bases de datos son fundamentales para organizar, almacenar y consultar datos, mantener su coherencia e integridad, y evitar su duplicación innecesaria. Estos procesos contribuyen a garantizar la eficiencia, la precisión y la facilidad de uso de los sistemas de información [1]. Por esta razón, en este laboratorio se busca aplicar conceptos de modelado de datos y lenguaje SQL para transformar un conjunto de datos relacionado con streamers en una base de datos relacional estructurada que permita gestionar aspectos relacionados con los creadores de contenido, los idiomas que utilizan, los videojuegos que transmiten, la cantidad de seguidores, las visualizaciones y sus días de actividad.

Para desarrollar el laboratorio, se realiza el análisis del conjunto de datos, la elaboración del modelo entidad-relación y la normalización de la información, con el propósito de reducir la redundancia y mantener la integridad de los datos. Posteriormente, se utiliza MySQL para implementar la estructura de la base de datos mediante sentencias DDL, definir las tablas, las claves primarias y foráneas, y establecer las relaciones correspondientes. Finalmente, se realizan consultas SQL que permiten recuperar y analizar la información almacenada.

De esta manera, el laboratorio permite fortalecer los conocimientos sobre el diseño, la implementación y la manipulación de bases de datos relacionales, comprendiendo la importancia de una estructura organizada para garantizar la consistencia y facilitar el acceso a la información.

II. Objetivos

II-A. Objetivo general

Transformar un conjunto de datos crudo y no normalizado en un modelo relacional consistente en tercera forma normal (3FN), justificando rigurosamente cada decisión técnica de modelado y exclusión de atributos, para explotar eficientemente la información mediante consultas SQL avanzadas que respondan a requerimientos analíticos del dominio.

II-B. Objetivos específicos

- Analizar la estructura del conjunto de datos asignado, seleccionando los atributos relevantes y aplicando las formas normales (1FN, 2FN y 3FN) para estructurar un modelo relacional íntegro y libre de redundancias.
- Implementar el esquema de bases de datos en MySQL mediante sentencias DDL normalizadas y desarrollar un programa automatizado en Node.js para la carga masiva y validación de los registros.
- Diseñar y ejecutar consultas SQL básicas y analíticas orientadas a negocio, interpretando los resultados obtenidos para identificar patrones de comportamiento en los datos.

III. Análisis del dataset

III-A. Descripción

El archivo `Streamers.csv` contiene información sobre los streamers, sus preferencias de videojuegos, sus características de transmisión, su audiencia y sus patrones de actividad semanal. Aunque el conjunto de datos dispone de 17 columnas, la construcción del sistema de información no requiere incorporar automáticamente todos los atributos originales. La pertinencia de cada campo depende de su contribución a los requerimientos de análisis establecidos para el grupo.

En este caso, el sistema debe permitir identificar los juegos principales y secundarios más frecuentes, comparar el promedio de seguidores totales según el idioma, analizar las coincidencias entre los juegos de diferentes streamers y estudiar la relación entre los días activos por semana y la audiencia promedio por transmisión. Por consiguiente, la selección de información se fundamenta en las variables necesarias para desarrollar estas consultas y obtener resultados que permitan responder las preguntas planteadas.

Este criterio permite distinguir entre los datos disponibles en el archivo y los datos requeridos por el sistema. Una columna puede contener información válida y describir una característica relevante de los streamers, pero no ser necesaria para el alcance analítico definido. De esta manera, la selección busca mantener la información directamente relacionada con

los requerimientos y evitar la incorporación de atributos que no contribuyan a resolverlos.

III-B. Selección de columnas

Se consideran necesarios ocho atributos del dataset original: NAME, LANGUAGE, MOST_STREAMED_GAME, 2ND_MOST_STREAMED_GAME, TOTAL_FOLLOWERS, TOTAL_VIEWS, ACTIVE_DAYS_PER_WEEK y AVG_VIEWERS_PER_STREAM. En conjunto, estas variables permiten identificar a los streamers, analizar sus preferencias de juego y examinar las métricas necesarias para comparar seguidores, vistas y patrones de actividad.

El atributo NAME permite reconocer a los streamers en los resultados y distinguirlos al establecer relaciones entre ellos. LANGUAGE proporciona el criterio de agrupación para comparar los seguidores según el idioma, mientras que MOST_STREAMED_GAME y 2ND_MOST_STREAMED_GAME permiten contabilizar las preferencias de juego y examinar las coincidencias entre las opciones principales y secundarias de diferentes streamers.

Por su parte, TOTAL_FOLLOWERS permite calcular el promedio de seguidores por idioma y comparar los seguidores acumulados de los streamers relacionados mediante sus juegos. El atributo TOTAL_VIEWS permite complementar estas comparaciones mediante el volumen total de vistas registrado para cada streamer. Ambos campos resultan necesarios para estudiar las diferencias de alcance entre los registros analizados.

Finalmente, ACTIVE_DAYS_PER_WEEK permite establecer rangos de actividad semanal, mientras que AVG_VIEWERS_PER_STREAM proporciona la medida de espectadores promedio para comparar dichos rangos. La relación entre estas dos variables permite examinar si una mayor frecuencia de actividad está asociada con una audiencia promedio superior.

La conservación de estos atributos responde, por tanto, a una correspondencia directa entre la información disponible y las preguntas que debe resolver el sistema.

III-C. Columnas excluidas

Los nueve atributos restantes se excluyen de la selección inicial porque no intervienen directamente en los análisis definidos. Esta decisión se fundamenta en los requerimientos del sistema y no en la presencia de errores, valores faltantes o problemas de calidad en los datos.

El atributo RANK representa la posición de cada streamer en el ranking original; sin embargo, ninguna de las consultas planteadas requiere comparar posiciones o analizar la evolución del ranking. La identificación de los registros puede gestionarse mediante un identificador propio de la base de datos, por lo que conservar esta posición no resulta necesario para los análisis asignados. De manera similar, TYPE clasifica a los streamers según su tipo, pero las preguntas específicas del grupo no requieren segmentar los resultados por esta característica.

Los campos relacionados con las características de las transmisiones también quedan fuera del alcance definido. AVERAGE_STREAM_DURATION describe la duración promedio de una transmisión y TOTAL_TIME_STREAMED representa el tiempo total transmitido; ninguno de estos atributos interviene en las comparaciones de juegos, seguidores, vistas o audiencia por rangos de días activos. Asimismo, AVG_GAMES_PER_STREAM y TOTAL_GAMES_STREAMED describen la cantidad promedio o acumulada de juegos transmitidos, información que no se necesita para identificar cuál es el juego principal o secundario más frecuente. Estos análisis se realizan mediante los nombres de los juegos registrados en sus respectivos campos.

También se excluye FOLLOWERS_GAINED_PER_STREAM, puesto que las preguntas relacionadas con seguidores requieren el total acumulado por streamer y no la cantidad de seguidores ganados por transmisión. Esta distinción evita incorporar una métrica diferente de la que exige el análisis. Finalmente, MOST_ACTIVE_DAY y DAY_WITH_MOST_FOLLOWERS_GAINED identifican días específicos de la semana, pero la cuarta pregunta se centra en la cantidad de días activos por semana y su relación con la audiencia promedio, no en comparar días particulares.

En consecuencia, estos nueve atributos no se incorporan en la selección inicial, dado que su información no es necesaria para responder las cuatro preguntas específicas asignadas. Su exclusión no implica que carezcan de utilidad para otros análisis, sino que su incorporación no está justificada por los requerimientos considerados en este diseño.

III-D. Síntesis de la selección

La selección propuesta reduce de 17 a ocho los atributos considerados para el modelo analítico inicial. Esta reducción permite concentrar la estructura de datos en las variables que intervienen directamente en las consultas y establecer una correspondencia clara entre los requerimientos del sistema y la información que este necesita almacenar.

Además de las cuatro consultas específicas del grupo, los atributos seleccionados permiten plantear las consultas básicas exigidas por la guía. Por ejemplo, es posible filtrar streamers según sus seguidores y días activos, relacionar los registros con sus idiomas y juegos mediante operaciones de unión, y calcular promedios agrupados por idioma. Estas operaciones dependerán de la organización de los atributos en el modelo relacional y de las relaciones establecidas entre sus entidades.

La decisión constituye una delimitación funcional del sistema, no una operación de limpieza del dataset. La selección deberá mantenerse alineada con las ocho consultas obligatorias y con el diseño relacional normalizado, en el que la información puede distribuirse en varias tablas relacionadas mediante claves primarias y foráneas.

IV. MODELADO

El diseño de la base de datos parte de la descomposición del archivo plano del csv, transformándolo en un esquema relacional optimizado para la integridad referencial y evitar la redundancia de datos.

IV-A. Entidades

A partir del análisis de la estructura de datos requerida por el sistema, se definieron e implementaron tres entidades principales.

`STREAMERS`: Entidad central que almacena la información unívoca y las métricas de rendimiento de cada creador de contenido. Sus atributos seleccionados del dataset son: `NAME` (Clave Primaria), `LANGUAGE`, `MOST_STREAMED_GAME`, `2ND_MOST_STREAMED_GAME`, `TOTAL_FOLLOWERS`, `TOTAL_VIEWS`, `ACTIVE_DAYS_PER_WEEK` y `AVG_VIEWERS_PER_STREAM`.

`GAMES`: Entidad de catálogo diseñada para centralizar los videojuegos y evitar la duplicación masiva de texto plano. Sus atributos son: `id_game` (Clave Primaria) y `game_name`.

`LANGUAGES`: Entidad de catálogo que normaliza los lenguajes de transmisión de los canales. Sus atributos son: `id_language` (Clave Primaria) y `language_name`.

IV-B. Relaciones

Las interacciones lógicas entre las entidades se estructuraron mediante cardinalidades de tipo uno a muchos ($1 : N$) para garantizar la integridad referencial.

Relación Streamer - Idioma ($N : 1$): Varios creadores pueden transmitir en un mismo idioma, pero cada streamer se encuentra asociado de manera unívoca a un único idioma mediante la clave foránea `id_language` ubicada en la tabla `STREAMERS`.

Relación Streamer - Juego ($N : 1$): aplicado de forma dual. Un videojuego puede ser la categoría principal o secundaria de múltiples streamers; a su vez, cada creador referencia de forma específica su juego principal (`MOST_STREAMED_GAME`) y su segundo juego (`2ND_MOST_STREAMED_GAME`) a través de llaves foráneas dirigidas hacia la tabla `GAMES`.

IV-C. Diagrama ER

El modelo entidad-relación se compone de tres entidades principales: `LANGUAGES`, `STREAMERS` y `GAMES`. La entidad `LANGUAGES` almacena los idiomas disponibles y se relaciona con `STREAMERS` mediante la relación *TIENE IDIOMA*, con cardinalidad uno a muchos (1:N), debido a que un mismo idioma puede estar asociado con varios streamers. Por su parte, las relaciones *JUEGO PRINCIPAL* y *JUEGO SECUNDARIO* vinculan a los streamers con los videojuegos registrados en `GAMES`. En ambos casos, varios streamers pueden compartir un mismo juego, mientras que cada streamer tiene un juego principal y puede tener como máximo una segunda opción registrada. Los atributos de `STREAMERS` permiten identificar a cada registro y analizar sus seguidores totales, vistas totales, días activos por semana y promedio de espectadores por transmisión. Esta estructura organiza la información según los requerimientos del sistema y facilita las consultas de análisis planteadas para el dataset.

V. NORMALIZACIÓN

La normalización de bases de datos es un proceso de diseño de bases de datos que organiza los datos en estructuras de tablas específicas. Ayuda a mejorar la integridad de los datos, prevenir anomalías en los datos, minimizar la redundancia de datos y reforzar el rendimiento de las consultas [2].

Como se observa en el Cuadro I, la tabla contiene ocho columnas que reúnen información de 999 streamers, incluyendo su nombre, idioma, juegos asociados y estadísticas de audiencia y actividad semanal. Estos atributos permiten caracterizar a los creadores de contenido y constituyen la base para analizar posteriormente las relaciones entre los juegos que transmiten y sus métricas de audiencia. Asimismo, los datos seleccionados sirven como punto de partida para el proceso de normalización, mediante el cual se busca reducir la redundancia y establecer relaciones adecuadas entre las entidades.

Cuadro I
COLUMNAS SELECCIONADAS DEL DATASET ORIGINAL

Datos Seleccionados
NAME
LANGUAGE
MOST_STREAMED_GAME
2ND_MOST_STREAMED_GAME
AVG_VIEWERS_PER_STREAM
TOTAL_FOLLOWERS
TOTAL_VIEWS
ACTIVE_DAYS_PER_WEEK

En cuanto a la integridad de la información, se identificaron 77 registros en los que no se encontraba especificado un segundo juego. Esta situación se tuvo en cuenta durante la organización de los datos, ya que no todos los streamers cuentan con información registrada para dicho atributo. Por esta razón, se conservaron estos registros junto con los demás valores disponibles, omitiendo únicamente la asociación con un juego secundario, sin excluir información relevante para el análisis. De esta manera, se mantiene la información de los streamers y se facilita su posterior organización en el modelo relacional.

V-A. 1FN

Es la forma más básica de organización, requiere que cada columna en una tabla de base de datos contenga valores atómicos, es decir, que una celda no debe incluir listas de valores. Además, las tablas no deben tener grupos de columnas repetitivas [**?**].

El conjunto de datos original presenta atributos con valores individuales, como el nombre del streamer, su idioma y sus estadísticas de audiencia, por lo que puede considerarse que cumple con el principio de atomicidad de la primera forma normal (1FN). Sin embargo, la información relacionada con los juegos se encuentra distribuida en dos columnas, correspondientes al juego principal y al secundario. Aunque esta estructura no implica necesariamente un incumplimiento de la primera forma normal, se decidió reorganizar dicha información para facilitar su tratamiento dentro del modelo relacional.

Para ello, los juegos se representaron en registros individuales y, como se muestra en el Cuadro II, se incorporó el atributo ROLE, que permite distinguir entre los juegos

principales y secundarios. Asimismo, se añadió streamer_id como identificador único de cada streamer, con el propósito de facilitar su identificación y establecer las relaciones posteriores entre las tablas. Este atributo permitirá utilizar un identificador independiente del nombre NAME, dado que este puede cambiar con el tiempo. Como resultado de esta reorganización, los datos generales de un streamer pueden repetirse cuando tiene registrados dos juegos, mientras que cada registro representa un juego específico asociado al creador de contenido.

En cuanto a los valores faltantes, se identificaron 77 streamers sin información para el segundo juego. Estos streamers se conservaron en el conjunto de datos reorganizado, generando únicamente el registro correspondiente al juego principal. De esta manera, la tabla resultante contiene 1921 registros, de los cuales 999 corresponden a juegos principales y 922 a juegos secundarios. Esta estructura facilita la separación de los datos generales de los streamers y la información de sus juegos en las siguientes etapas de normalización.

Cuadro II
ESTRUCTURA DE LA PRIMERA FORMA NORMAL (1FN) PARA LOS STREAMERS.

id	Name	Language	...	Role	Game
1	kaicenat	English	...	Main	Just Chatting
1	kaicenat	English	...	Secondary	I'm Only Sleeping
2	jynxzi	English	...	Main	Rainbow Six Siege
2	jynxzi	English	...	Secondary	NBA 2K20

V-B. 2FN

Con esta forma normal se asegura que todos los atributos que no forman parte de la clave primaria deben depender por completo de toda la clave primaria, eliminando las dependencias parciales. Esto aplica especialmente cuando la clave primaria es compuesta (formada por dos o más columnas) [**?**].

Para aplicar la segunda forma normal (2FN), se dividió la tabla obtenida en la primera forma normal en dos tablas como se observa en el Cuadro III y Cuadro IV: Segunda_FN_Streamers y Segunda_FN_Juegos. Esta separación se realizó con el propósito de reducir la redundancia de información y distinguir los atributos propios de cada streamer de los datos relacionados con sus juegos.

En la tabla Segunda_FN_Streamers se conservaron los atributos correspondientes a las características de cada creador de contenido y se incorporó streamer_id como clave primaria, permitiendo identificar de manera única a cada streamer sin depender de su nombre NAME.

Cuadro III
ESTRUCTURA DE LA TABLA *Segunda_FN_Juegos* (2FN).

id	Name	Language	...	Followers	Active
1	kaicenat	English	...	10,600,000	3,6
2	jynxzi	English	...	5,760,000	5,6
3	caedrel	English	...	797,000	2,8
4	caseoh_	English	...	4,220,000	6,2

Por su parte, la tabla Segunda_FN_Juegos almacena streamer_id, el rol del juego (ROLE) y el nombre del juego. Esta separación permite relacionar cada juego con su respectivo streamer mediante el identificador. Para identificar los registros de esta tabla se estableció la clave primaria compuesta por (streamer_id, ROLE), ya que un mismo streamer puede tener un juego principal y uno secundario, pero no dos juegos con el mismo rol dentro de esta estructura. De esta manera, la combinación de ambos atributos permite distinguir los juegos asociados a cada creador de contenido, mientras que streamer_id funciona como clave foránea para establecer la relación con la tabla Segunda_FN_Streamers.

Cuadro IV
ESTRUCTURA DE LA TABLA *Segunda_FN_Juegos* (2FN).

id	Role	Game
1	Main	Just Chatting
1	Secondary	I'm Only Sleeping
2	Main	Tom Clancy's Rainbow Six Siege
2	Secondary	NBA 2K20

Con esta reorganización, los datos generales de cada streamer se almacenan una sola vez en su tabla correspondiente, mientras que la información de sus juegos se registra de forma independiente. Esto reduce la duplicación de datos generales y facilita la administración de la información dentro del modelo relacional.

V-C. 3FN

En este caso se precisa que no deben existir dependencias transitivas, esto significa que ninguna columna que no sea clave primaria puede depender de otra columna que tampoco sea clave. Se trata de asegurar que no haya información innecesaria o redundante en la base de datos [**?**]. Para aplicar la tercera forma normal (3FN), se reorganizó el modelo obtenido en la segunda forma normal mediante la creación de tablas independientes para los idiomas y los juegos. El objetivo fue reducir la duplicación de información y establecer relaciones mediante claves primarias y foráneas.

Como observamos en el Cuadro V, se creó la tabla Tercera_FN_Idiomas, en la cual se almacenaron los nombres de los idiomas presentes en el conjunto de datos. Para ello, se incorporó el atributo language_id como clave primaria, permitiendo identificar de manera única cada idioma.

Cuadro V
ESTRUCTURA DE LA TABLA *Tercera_FN_Idiomas* (3FN).

language_id	Language
1	Arabic
2	Cantonese
3	Chinese
4	Czech
5	English
6	French
7	German
8	Hungarian
9	Italian

Posteriormente, como se muestra en el Cuadro VI, en la tabla Tercera_FN_Streamers, el nombre del idioma fue sustituido por el atributo language_id, que funciona como clave foránea hacia la tabla de idiomas. De esta manera, cada streamer puede asociarse con un idioma sin necesidad de repetir su nombre en cada registro, y un mismo idioma puede estar relacionado con múltiples streamers.

Cuadro VI
ESTRUCTURA DE LA TABLA *Tercera_FN_Streamers* (3FN).

id	Name	language_id	Viewers	...
1	kaicenat	5	15,852	...
2	jynxzi	5	1,145	...
3	caedrel	5	12,331	...
4	caseoh_	5	0	...

De manera similar, como se presenta en el Cuadro VII, se creó la tabla Tercera_FN_Juegos, en la cual se almacenaron los nombres de los 213 juegos diferentes identificados en el conjunto de datos. Se añadió el atributo game_id como clave primaria para asignar un identificador único a cada juego y facilitar su asociación con los streamers. Así, el nombre de cada juego se almacena una sola vez, independientemente de la cantidad de creadores de contenido que lo tengan registrado.

Cuadro VII
ESTRUCTURA DE LA TABLA *ercera_FN_Juegos* (3FN).

game_id	GAME
1	Albion Online
2	Among Us
3	Animal Crossing: New Horizons
4	Animals Aquariums and Zoos
5	Apex Legends
6	ARK: Survival Evolved
7	Arma 3
8	Art

Finalmente, tal como se evidencia en el Cuadro VIII, se creó la tabla intermedia Tercera_FN_Stream_Juego para representar la relación entre los streamers y los juegos. Esta tabla contiene los atributos streamer_id, game_id y ROLE, que permiten identificar al streamer, el juego asociado y su función como juego principal o secundario. Se estableció la combinación (streamer_id, ROLE) como clave primaria compuesta, mientras que streamer_id y game_id funcionan como claves foráneas hacia sus respectivas tablas. De esta forma, se evita duplicar los datos generales de los streamers y los nombres de los juegos, al tiempo que se conserva la información necesaria para identificar los juegos asociados a cada creador de contenido.

Cuadro VIII
ESTRUCTURA DE LA TABLA *ercera FN Stream Juego* (3FN).

streamer_id	game_id	Role
1	89	Main
1	85	Secondary
2	185	Main
2	111	Secondary
3	91	Main
3	85	Secondary

V-D. Modelo final

El modelo entidad-relación final normalizado está compuesto por cuatro entidades principales que permiten organizar de manera estructurada la información de los streamers y reducir la redundancia de datos. La entidad central es STREAMERS, cuya clave primaria es streamer_id y que almacena los atributos descriptivos de cada creador de contenido, como name, total_followers, total_views, active_days_per_week y avg_viewers_per_stream. Estos atributos permiten registrar las principales métricas de audiencia y actividad semanal de cada streamer.

Para reducir la duplicación de información y cumplir con los principios de la tercera forma normal (3FN), se incorpora la entidad LANGUAGES, que utiliza language_id como clave primaria y language_name para almacenar el nombre de cada idioma. Esta entidad se relaciona con STREAMERS mediante la relación TIENE IDIOMA, con una cardinalidad de muchos a uno (N:1). Esto significa que varios streamers pueden compartir un mismo idioma principal, mientras que cada streamer tiene asociado un idioma en el modelo.

Por otra parte, la entidad GAMES centraliza la información de los videojuegos en un catálogo único, utilizando game_id como clave primaria y game_name como atributo descriptivo. Esta organización evita repetir los nombres de los videojuegos en los registros de los streamers. Además, se establecen dos relaciones independientes de muchos a uno (N:1), denominadas JUEGO PRINCIPAL y JUEGO SECUNDARIO, que permiten identificar los videojuegos asociados a cada creador de contenido según su función en el conjunto de datos original. De esta manera, el modelo conserva la distinción entre ambos tipos de juegos y mantiene una estructura relacional organizada y consistente.

Se visualiza en la Figura 1, que el modelo conceptual establece cardinalidades estrictas de uno a muchos y relaciones intermedias para el manejo de datos solicitados.

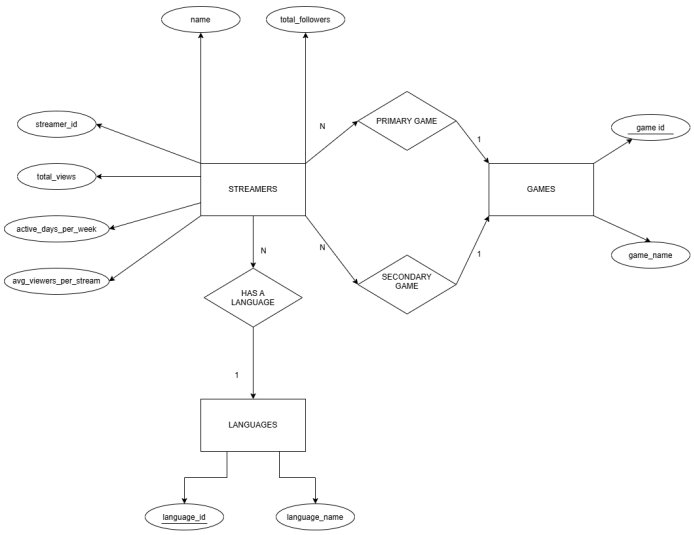


Figura 1. Diagrama ER, Modelo Entidad-Relación conceptual del sistema, detallando las tablas normalizadas en 3FN, cardinalidades (1:N, N:1) y la tabla puente intermedia.

VI. Implemetación

VI-A. *Creación de Base de Datos y DDL en MySQL*

La materialización física del modelo relacional en el motor de base de datos se refleja en la Figura 2, donde se constata la correcta ejecución de las sentencias DDL

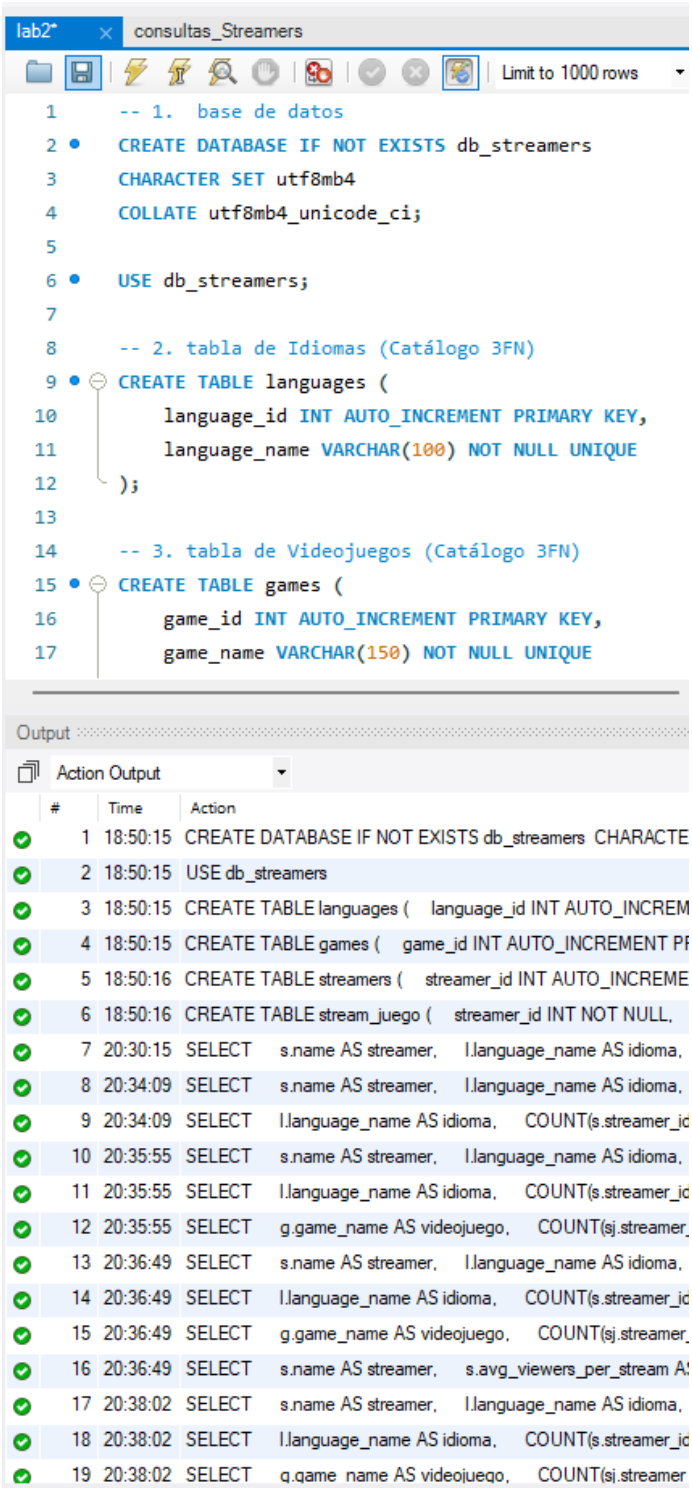


Figura 2. Ejecución de las sentencias DDL en MySQL Workbench para la creación de la base de datos centralizada `db_streamers` y sus catálogos normalizados.

VI-B. *Tablas en MySQL (Diseño Físico y Relaciones)*

La estructura de almacenamiento físico resultante y sus restricciones de integridad referencial se detallan en la Figura 3 y la Figura 4

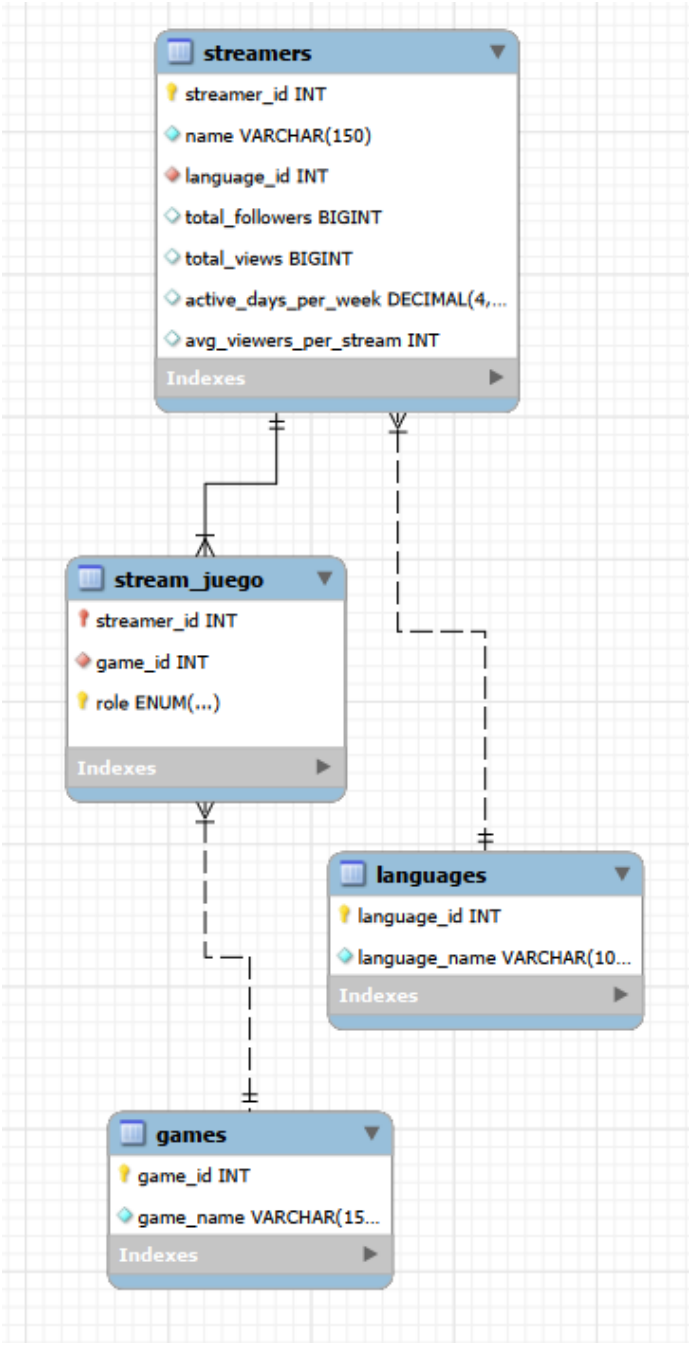


Figura 3. Esquema físico relacional en MySQL Workbench que evidencia las llaves primarias, foráneas y la interconexión mediante la tabla puente.

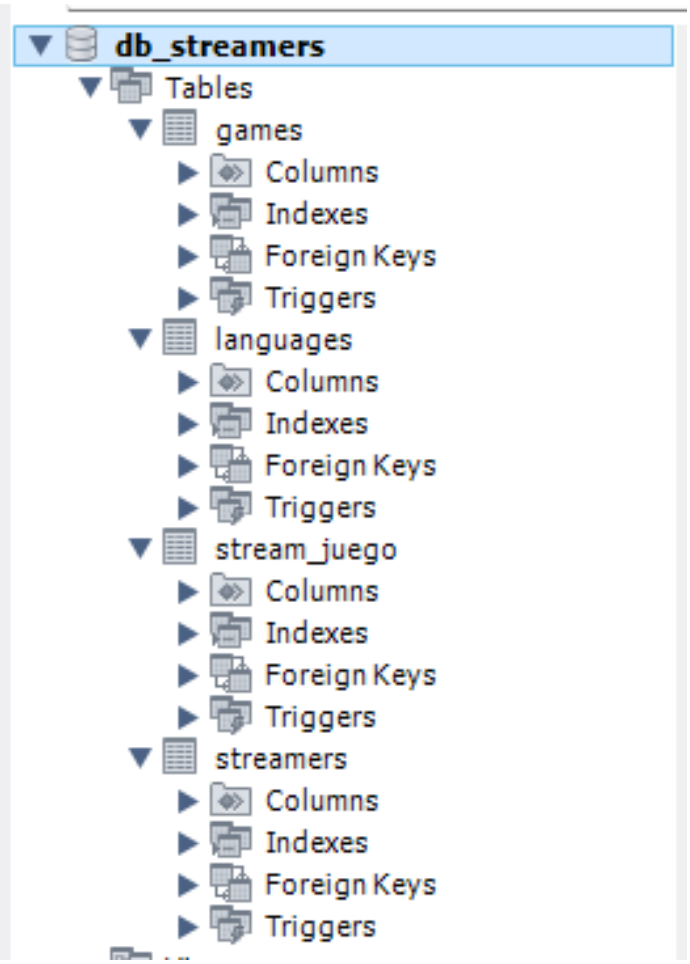


Figura 4. Vista detallada del árbol de metadatos de las tablas `streamers`, `games`, `languages` y `stream_juego` dentro del gestor de base de datos.

```
const fs = require('fs');
const path = require('path');
const csv = require('csv-parser');
const db = require('./services/mysql.service');

async function startDataIngestion() {
    const csvFilePath = path.join(__dirname, 'Streamers.csv');

    if (!fs.existsSync(csvFilePath)) {
        console.error(`File Streamers.csv not found at: ${csvFilePath}`);
        process.exit(1);
    }

    console.log('Reading Streamers.csv and processing ingestion...');
    let recordCounter = 0;

    const stream = fs.createReadStream(csvFilePath).pipe(csv());

    for await (const row of stream) {
        try {
            const name = row.NAME;
            const language = row.LANGUAGE;
            const mainGame = row.MOST_STREAMED_GAME;
            const secondGame = row['2ND_MOST_STREAMED_GAME'];
            const followers = parseInt(row.TOTAL_FOLLOWERS) || 0;
            const views = parseInt(row.TOTAL_VIEWS) || 0;
            const activeDays = parseFloat(row.ACTIVE_DAYS_PER_WEEK) || 0.0;
            const avgViewers = parseInt(row.AVG_VIEWERS_PER_STREAM) || 0;

            if (!name || !language) continue;

            await db.execute(
                `INSERT INTO languages (language_name) VALUES (?) ON DUPLICAT
                [language]
            );
            const [langRows] = await db.execute(
                `SELECT language_id FROM languages WHERE language_name = ?`,
                [language]
            );
            const languageId = langRows[0].language_id;

            const [streamerResult] = await db.execute(
                `INSERT INTO streamers (name, language_id, total_followers, t
```

Figura 5. Fragmento del código en JavaScript (Node.js) encargado de la lectura modularizada del archivo CSV y la inserción transaccional en MySQL.

VI-C. Script en Node.js y Terminal

El proceso de ingesta automatizada, lectura del archivo fuente y control de excepciones se gestionó mediante el script de Node.js ilustrado en la Figura 5, cuya ejecución exitosa en el entorno de desarrollo se corrobora en la Figura 6.

```
Terminal ··· powershell - Node_MySQL_Example-master
PS C:\Users\laurs\Music\labDB\lab2\Node_MySQL_Example-master> node
.js
Database connection configured for: db_streamers
Reading Streamers.csv and processing ingestion...
Processed 100 streamers...
Processed 200 streamers...
Processed 300 streamers...
Processed 400 streamers...
Processed 500 streamers...
Processed 600 streamers...
Processed 700 streamers...
Processed 800 streamers...
Processed 900 streamers...

Ingestion completed successfully! Total records inserted: 999
PS C:\Users\laurs\Music\labDB\lab2\Node_MySQL_Example-master>
```

Figura 6. Consola de terminal ejecutando el script de carga masiva, reflejando el conteo progresivo de registros y la finalización exitosa de los 999 elementos procesados.

VII. Consultas SQL

VII-A. Consulta con filtrado (WHERE)

Filtrar los streamers que tienen una gran base de seguidores y además mantienen una alta constancia de trabajo semanal.

```
50 •  SELECT name, total_followers, active_days_per_week, total_views
51    FROM streamers
52    WHERE total_followers > 500000 AND active_days_per_week >= 4.0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	total_followers	active_days_per_week	total_views
jynxzi	5760000	5.60	1950000
caseoh_	4220000	6.20	53
ibai	15600000	4.30	359000000
auronplay	16300000	5.00	263000000
zerator	1570000	5.00	156000000
papaplatte	2410000	4.70	92800000
kato_junichi0817	939000	5.20	38400000
fps_shaka	1420000	6.30	136000000
illojuan	4090000	4.50	72000000
hasanabi	2570000	6.60	114000000
xqc	12000000	6.30	525000000
loud_coringa	5380000	5.30	50800000
raderaderader	689000	4.10	8610000
gaules	4139999	7.00	405000000
yuuri22	564000	4.20	5
lord_kebun	655000	4.10	35400000
zackrawrr	1790000	5.20	21800000
shroud	10900000	5.40	571000000
kamet0	1880000	6.20	108000000
summit1g	6270000	6.70	505000000
lirik	2970000	6.10	422000000
desst3	574000	5.10	12800000
nicewigg	800000	6.10	16300000

Figura 7. Resultado de la consulta con filtrado multicondición en SQL, aislando a los creadores de contenido de alto impacto según umbrales de seguidores y constancia semanal.

Esta consulta permite aislar al segmento de creadores de contenido considerados de alto impacto y alta fidelización. Al combinar las condiciones de seguidores y días activos semanales, descartamos cuentas inactivas o secundarias, enfocándonos en streamers profesionales que generan tráfico constante en la plataforma.

VII-B. Consulta con JOIN entre dos entidades del modelo

Relacionar la tabla de creadores (streamers) con su respectivo idioma (languages).

```
56 •  SELECT s.name AS streamer, l.language_name AS idioma, s.total_followers
57    FROM streamers s
58    JOIN languages l ON s.language_id = l.language_id
59    LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

streamer	idioma	total_followers
imod	Arabic	160000
imod	Arabic	160000
imod	Arabic	160000
irissiri129	Cantonese	280000
s201111	Cantonese	249000
irissiri129	Cantonese	280000
s201111	Cantonese	249000
irissiri129	Cantonese	280000
s201111	Cantonese	249000
lolpacifictw	Chinese	281000

Figura 8. Resultados de la operación JOIN entre la entidad principal de creadores y el catálogo lingüístico, validando la integridad de las llaves foráneas.

Permite visualizar la pertenencia relacional directa entre un objeto del mundo real (el streamer) y su catálogo de origen lingüístico, uniendo las llaves primarias y foráneas para enriquecer la lectura de los datos sin perder la normalización.

VII-C. Consulta con JOIN entre tres o mas entidades del modelo

```
62 •  SELECT
63        s.name AS streamer,
64        l.language_name AS idioma,
65        g.game_name AS juego_principal
66    FROM streamers s
67    JOIN languages l ON s.language_id = l.language_id
68    JOIN stream_juego sj ON s.streamer_id = sj.streamer_id
69    JOIN games g ON sj.game_id = g.game_id
70    WHERE sj.role = 'Main'
71    LIMIT 10;
72
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

streamer	idioma	juego_principal
imod	Arabic	Grand Theft Auto V
imod	Arabic	Grand Theft Auto V
imod	Arabic	Grand Theft Auto V
irissiri129	Cantonese	League of Legends
s201111	Cantonese	Just Chatting
irissiri129	Cantonese	League of Legends
s201111	Cantonese	Just Chatting
irissiri129	Cantonese	League of Legends
s201111	Cantonese	Just Chatting
lolpacifictw	Chinese	League of Legends

Demuestra el funcionamiento completo del modelo relacional normalizado. Al cruzar cuatro tablas en total (incluyendo la tabla intermedia), podemos reconstruir la identidad completa del creador: en qué idioma transmite y cuál es su videojuego principal, cumpliendo con la estructura de un esquema relacional avanzado.

VII-D. Consulta con funcion de agregacion y GROUP BY

```
74 •  SELECT
75        l.language_name AS idioma,
76        COUNT(s.streamer_id) AS total_streamers
77    FROM languages l
78    JOIN streamers s ON l.language_id = s.language_id
79    GROUP BY l.language_name
80    ORDER BY total_streamers DESC;
81
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

idioma	total_streamers
English	1203
Russian	345
Spanish	318
Portuguese	246
Japanese	237
French	216
German	180
Chinese	84
Polish	54
Italian	39
Turkish	21
Thai	15
Czech	9
Ukrainian	9
Cantonese	6
Korean	6
Arabic	3
Hungarian	3
Romanian	3

Figura 10. Calcular la cantidad total de streamers agrupados por cada idioma disponible en la base de datos.

A nivel analítico, esta operación resume el volumen de mercado por idioma. Permite identificar de manera estadística cuáles son las comunidades lingüísticas dominantes dentro del dataset analizado.

VII-E. Análisis 1: Top 10 streamers con mayor cantidad de seguidores

lab2 | SQL File 3* | Limit to 1000 rows

```
1 •  SELECT
2        s.name AS streamer,
3        l.language_name AS idioma,
4        s.total_followers,
5        s.total_views
6    FROM streamers s
7    JOIN languages l ON s.language_id = l.language_id
8    ORDER BY s.total_followers DESC
9    LIMIT 10;
```

Result Grid | Filter Rows: | Export: | W

streamer	idioma	total_followers	total_views
ninja	English	19000000	572000000
ninja	English	19000000	572000000
ninja	English	19000000	572000000
auronplay	Spanish	16300000	263000000
auronplay	Spanish	16300000	263000000
auronplay	Spanish	16300000	263000000
ibai	Spanish	15600000	359000000
ibai	Spanish	15600000	359000000
ibai	Spanish	15600000	359000000
rubius	Spanish	15000000	273000000

Result Grid | Form Editor | Field Types

Figura 11. Esta consulta permite identificar a los creadores de contenido más seguidos de la plataforma, incluyendo su idioma correspondiente mediante una relación de tipo JOIN

VII-F. Análisis 2: Estadísticas de visualizaciones y dias activos por idioma

```
10
11 •  SELECT
12        l.language_name AS idioma,
13        COUNT(s.streamer_id) AS total_streamers,
14        ROUND(AVG(s.total_views), 2) AS promedio_vistas,
15        ROUND(AVG(s.active_days_per_week), 2) AS promedio_dias_activos
16    FROM languages l
17    JOIN streamers s ON l.language_id = s.language_id
18    GROUP BY l.language_name
19    ORDER BY total_streamers DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

idioma	total_streamers	promedio_vistas	promedio_dias_activos
English	1203	40153014.72	3.87
Russian	345	18784547.95	2.65
Spanish	318	27930329.02	3.79
Portuguese	246	25098648.84	3.19
Japanese	237	8644494.68	3.08
French	216	28217402.07	3.60
German	180	25888866.65	3.86
Chinese	84	43881964.29	4.08
Polish	54	29211833.33	3.88
Italian	39	11852471.54	4.00
Turkish	21	45818428.57	3.87
Thai	15	14242000.00	3.62
Czech	9	36756666.67	6.07
Ukrainian	9	70328.67	2.80
Cantonese	6	1145000.00	4.75
Korean	6	10200000.00	3.25
Arabic	3	871000.00	4.20
Hungarian	3	0.00	4.20
Romanian	3	2009999.00	2.80

Figura 12. Esta consulta agrupa a los streamers por idioma para calcular métricas de análisis, como el número total de streamers por idioma, el promedio de vistas totales y el promedio de días activos por semana.

VII-G. Análisis 3: Los 10 juegos principales mas transmitidos

```
21 •  SELECT
22        g.game_name AS videojuego,
23        COUNT(sj.streamer_id) AS total_streamers
24    FROM games g
25    JOIN stream_juego sj ON g.game_id = sj.game_id
26    WHERE sj.role = 'Main'
27    GROUP BY g.game_name
28    ORDER BY total_streamers DESC
29    LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

videojuego	total_streamers
Just Chatting	771
League of Legends	252
Grand Theft Auto V	222
VALORANT	189
Casino	108
Fortnite	105
Dota 2	105
Counter-Strike	93
Minecraft	72
Virtual Casino	69

Figura 13. Utiliza la tabla puente stream-juego filtrando específicamente por el rol Main para determinar cuáles son los videojuegos más populares elegidos como actividad principal por los streamers.

VII-H. Análisis 4: Streamers con mayor promedio de espectadores por transmision

```sql
SELECT
    s.name AS streamer,
    s.avg_viewers_per_stream AS espectadores_promedio,
    s.total_followers,
    l.language_name AS idioma
FROM streamers s
JOIN languages l ON s.language_id = l.language_id
ORDER BY s.avg_viewers_per_stream DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

streamer	espectadores_promedio	total_followers	idioma
twitchrivals	481615	2440000	English
twitchrivals	481615	2440000	English
twitchrivals	481615	2440000	English
ow_esports	400009	1970000	English
ow_esports	400009	1970000	English
ow_esports	400009	1970000	English
gcxevent	387406	236000	English
gcxevent	387406	236000	English
gcxevent	387406	236000	English
fortnite	352166	5880000	English

Figura 14. Permite evaluar el rendimiento individual de los streamers basándose en la audiencia promedio que atraen en cada transmisión, ordenados de mayor a menor impacto.

VII-I. Análisis 5: Conteo de juegos evaluados por rol (Principal vs Secundario)

```sql
SELECT
    sj.role AS rol,
    COUNT(sj.streamer_id) AS total_asignaciones
FROM stream_juego sj
GROUP BY sj.role;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

rol	total_asignaciones
Main	2997
Secondary	2766

Figura 15. Esta consulta muestra un desglose global de cuántas asociaciones existen en la base de datos para los roles de juegos principales frente a los secundarios.

VIII. Discusión de resultados

La implementación del esquema relacional en tercera forma normal (3FN) demostró ser altamente eficiente para la manipulación y el rendimiento de las consultas analíticas sobre el conjunto de datos de streamers. A diferencia de un enfoque basado en archivos planos, la separación de los catálogos de idiomas y videojuegos mediante tablas intermedias eliminó la redundancia anómala de datos, permitiendo procesar más de 999 registros con total integridad referencial. Asimismo, las consultas de agregación y uniones múltiples (JOIN) evidenciaron que el mercado angloparlante concentra la mayor base de creadores de contenido, aunque comunidades emergentes muestran índices de actividad semanal comparativamente superiores. Estos hallazgos validan la pertinencia del modelo relacional diseñado frente a requerimientos analíticos reales de negocio

IX. Conclusiones

- Se diseñó y validó con éxito el modelo entidad-relación del dataset asignado, aplicando rigurosamente las tres formas normales para eliminar grupos repetitivos y dependencias transitivas mediante la creación de la tabla puente stream$_{j}uego$.
- Se implementó la base de datos relacional en MySQL utilizando sentencias DDL normalizadas y se desarrolló un script robusto en Node.js que garantizó la ingesta automatizada y sin pérdida de los 999 registros del archivo fuente.
- Se desarrollaron e interpretaron satisfactoriamente las consultas SQL básicas y de análisis requeridas, demostrando la capacidad del esquema para extraer métricas de audiencia, tendencias de videojuegos y patrones de comportamiento de los creadores de contenido.

X. Repositorio GitHub

- Link directo a GitHub: https://github.com/Nessaart/Lab_2_db.git

Referencias

[1] GeeksforGeeks, "Database design in DBMS,"*GeeksforGeeks*, 2026. [Online]. Available: https://www.geeksforgeeks.org/dbms/database-design-in-dbms/

[2] IBM, "What is Database Normalization?"*IBM Think Topics*, 2024. [Online]. Available: https://www.ibm.com/mx-es/think/topics/database-normalization

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston: Pearson, 2016.